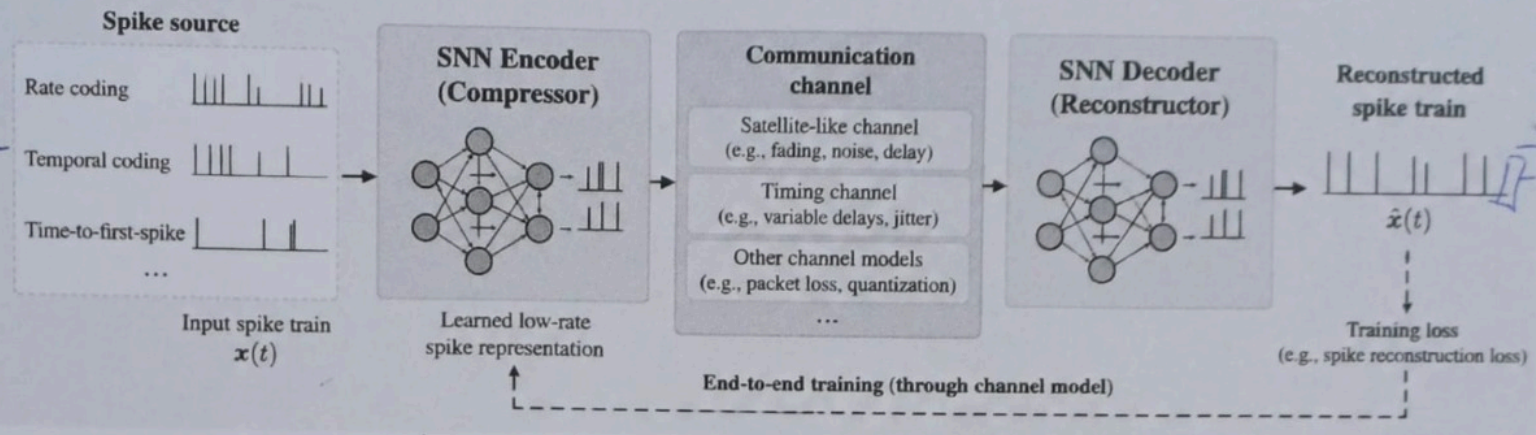
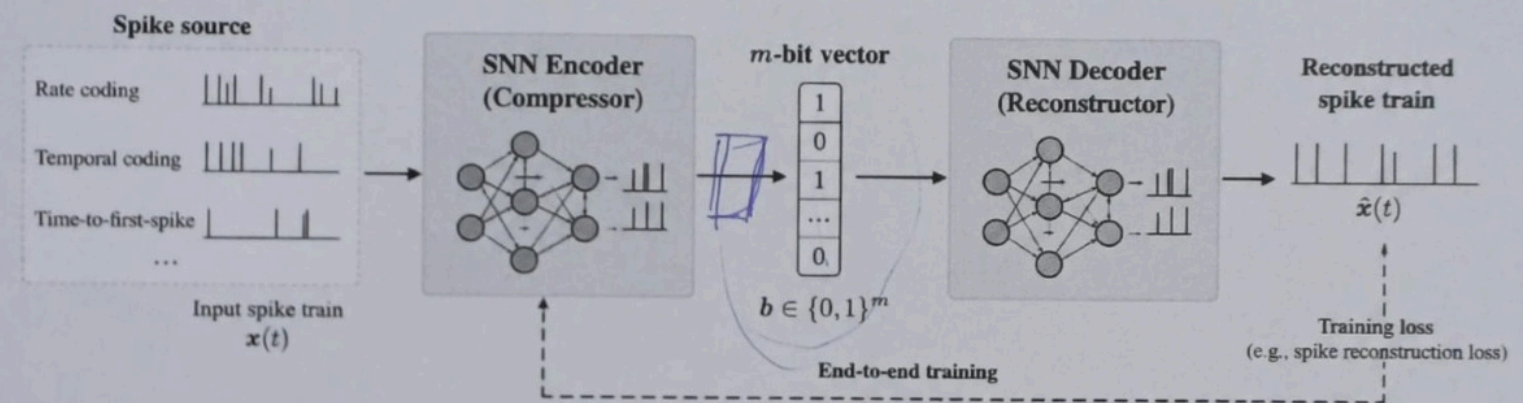


Task 1: End-to-end learned spike-train compression with channel



Task 2: End-to-end learned spike-train compression to m -bit vector (no channel)



Project: Spike Train Compression for Distributed Neuromorphic Detection

1 End-to-end learned spike-train compression with channel

Implement in Python an end-to-end communication system in which an SNN-based encoder compresses an input spike train, the compressed representation is transmitted over a communication channel, and an SNN-based decoder reconstructs the original spike train. Use different spike sources (e.g., rate coding, temporal coding, time-to-first-spike coding) and different channel models, in particular models inspired by satellite communications as well as timing-channel models.

2 End-to-end learned compression to m bits (no channel)

Implement a similar end-to-end system, but without the channel. The encoder should output a fixed-length m -bit vector (or a short bit sequence) that represents the input spike train. The decoder, implemented as an SNN, reconstructs the spike train from this compact representation. Investigate the tradeoff between the number of bits m and the reconstruction quality for different spike sources.

3 Extension to distributed neuromorphic detection/classification (optional)

If time permits, extend the framework to a task-oriented setting. Instead of (or in addition to) reconstructing the remote spike train, train the system for detection or classification. Allow the decoder to also have its own local spike observations, leading to a distributed setting with correlated neuromorphic data. Study how much information must be communicated to maintain reliable inference performance.